

# Contents

**Experiment-Overview: TDD-Workflows × Modelle × Prompt-Stile**

1. Forschungsfragen-Übersicht . . . . .

2. Experiment-Design . . . . .

2.1 Variablen . . . . .

2.2 Workflow → Prompt-Mapping . . . . .

3. Methodik . . . . .

3.1 Run-Pipeline . . . . .

3.2 Erfasste Metriken . . . . .

3.3 Bewertungsgrundsätze . . . . .

4. Ergebnisse . . . . .

4.1 RQ-1 — Wirkt der gewählte Workflow auf Code-Qualität, Korrektheit und TDD-Disziplin?

4.2 RQ-2 — Wirkt Prompt-Stil (prose / example-mapping / user-story) auf Code-Qualität und Korrektheit?

4.3 RQ-3 — Wirken Modell-Klasse (Opus / Sonnet / Haiku) und Thinking-Mode auf Output-Qualität und Effizienz?

4.4 RQ-4 — Profitieren schwächere Modelle stärker von strikteren Workflows als starke?

4.5 RQ-5 — Wie groß ist die Run-zu-Run-Varianz innerhalb identischer Zellen?

5. Cross-RQ-Synthese . . . . .

6. Caveats — revidierte / verworfene / nicht-prüfbare Befunde . . . . .

[!] Revidiert . . . . .

[X] Nicht prüfbar . . . . .

7. Limitierungen . . . . .

8. Reproduzierbarkeit . . . . .

9. Files . . . . .

## Experiment-Overview: TDD-Workflows × Modelle × Prompt-Stile

Stand: 2026-05-04. Datenbasis: experiments/runs/ (97 Runs gesamt).

Diese Studie untersucht den Effekt von TDD-Workflow-Strenges, Modell-Klasse, Thinking-Modus und Prompt-Stil auf Code-Qualität, Korrektheit und TDD-Disziplin bei kleinen TypeScript-Katas. Der Snapshot fasst den Stand der fünf aktiven Forschungsfragen (RQ-1 bis RQ-5) zum Stichtag zusammen — basiert ausschließlich auf den persistenten Befund-Listen unter research/RQ-\*/findings.md und ersetzt diese nicht, sondern friert eine reviewbare Querschnitts-Sicht ein.

### 1. Forschungsfragen-Übersicht

| RQ   | Frage                                                                         | Status | Cells | Coverage    | n Runs |
|------|-------------------------------------------------------------------------------|--------|-------|-------------|--------|
| RQ-1 | Wirkt der gewählte Workflow auf Code-Qualität, Korrektheit und TDD-Disziplin? | aktiv  | 5     | 5/5 (100 %) | 30     |

| RQ   | Frage                                                                                             | Status | Cells | Coverage      | n Runs |
|------|---------------------------------------------------------------------------------------------------|--------|-------|---------------|--------|
| RQ-2 | Wirkt Prompt-Stil (prose / example-mapping / user-story) auf Code-Qualität und Korrektheit?       | aktiv  | 3     | 3/3 (100 %)   | 13     |
| RQ-3 | Wirken Modell-Klasse (Opus / Sonnet / Haiku) und Thinking-Mode auf Output-Qualität und Effizienz? | aktiv  | 5     | 5/5 (100 %)   | 18     |
| RQ-4 | Profitieren schwächere Modelle stärker von strikteren Workflows als starke?                       | aktiv  | 12    | 12/12 (100 %) | 48     |
| RQ-5 | Wie groß ist die Run-zu-Run-Varianz innerhalb identischer Zellen?                                 | aktiv  | 11    | 11/11 (100 %) | 50     |

## 2. Experiment-Design

### 2.1 Variablen

**Workflow** — fünf Klassen mit zunehmender TDD-Strengung:

| Workflow           | Aufbau                                                      | TDD-Strengung            |
|--------------------|-------------------------------------------------------------|--------------------------|
| v1-oneshot         | "Implementiere X."                                          | keine                    |
| v2-iterative       | "Plane Schritt für Schritt, dann implementiere."            | keine                    |
| v3-basic-tdd       | "Verwende TDD."                                             | minimal (Self-Reporting) |
| v4-exact-subagents | Eigener Subagent pro Phase (Predictor + Red/Green/Refactor) | strikt, multi-context    |

| Workflow                | Aufbau                                                    | TDD-Strenge            |
|-------------------------|-----------------------------------------------------------|------------------------|
| v5-exact-single-context | Alle Phasen in einer Konversation, gleiches Phasen-Skript | strikt, single-context |

Konfiguration: experiments/workflows/v{1..5}-\*/.claude/agents/ und .claude/rules/.

**Modell x Thinking** (Lab-Varianten-IDs):

| Lab-Varianten-ID       | API-ID                    | Thinking |
|------------------------|---------------------------|----------|
| opus-4-7               | claude-opus-4-7           | Adaptive |
| opus-4-7-no-thinking   | claude-opus-4-7           | aus      |
| sonnet-4-6             | claude-sonnet-4-6         | Extended |
| sonnet-4-6-no-thinking | claude-sonnet-4-6         | aus      |
| haiku-4-5              | claude-haiku-4-5-20251001 | Extended |

**Kata x Prompt-Stil** (aktive Katas):

| Kata         | Prompt-Stile                                        | Komplexität      |
|--------------|-----------------------------------------------------|------------------|
| game-of-life | prose, example-mapping, user-story                  | groß (~40 LoC)   |
| mars-rover   | prose, (example-mapping, user-story selten erhoben) | mittel (~30 LoC) |

Prompt-Stile: - **prose**: Beschreibung der Regeln in Prosa, keine Test-Beispiele. - **example-mapping**: Regel + 1–3 konkrete Input/Output-Beispiele pro Regel. - **user-story**: “Als X möchte ich Y, damit Z” — Beschreibung ohne Beispiele.

**2.2 Workflow → Prompt-Mapping**

Aus methodischer Symmetrie (siehe research/README.md):

| Workflow   | erlaubte Prompt-Stile | Begründung                                                                                                                        |
|------------|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| v1, v2     | nur prose             | Test-Beispiele in example-mapping wären für Non-TDD-Workflows ein verstecktes Test-Geschenk → unfair gegenüber den TDD-Workflows. |
| v3, v4, v5 | alle drei             | Beispiele dienen als natürliche Test-Cases — für TDD-Workflows ist das das Idealbild der Aufgabe.                                 |

**3. Methodik**

Pipeline-Status zum Stichtag verifiziert gegen experiments/docker/Dockerfile, experiments/analyze-run.sh und experiments/aggregate-by-query.py: unverändert seit dem v2-Snapshot — claude-code-Pin auf 2.1.107, gawk im Image, RQ-Aggregation per Selektor-Query auf experiments/runs/.

### 3.1 Run-Pipeline

1. Container-Image docker-batch (Node 22 slim, claude-code 2.1.107 gepinnt) wird gestartet.
2. Run-Dir runs/<timestamp>\_<kata>\_<workflow>\_<model>/ wird angelegt; Workflow-Konfig (.claude/agents/, .claude/rules/) und Kata-Prompt (prompt.md) hinein kopiert.
3. pnpm-Workspace mit TypeScript, Vitest, ESLint+SonarJS aufgesetzt.
4. claude --print "\$(< prompt.md)" läuft headless, ohne HITL.
5. analyze-run.sh schreibt metrics.json und analysis-report.md.
6. aggregate-by-query.py <RQ>/ baut runs.csv und summary.md pro RQ.

### 3.2 Erfasste Metriken

**Korrektheit:** tests\_passing. **Effizienz:** duration\_seconds, total\_tokens, context\_utilization\_pct. **Code-Volumen:** lines\_of\_code, test\_lines, tests\_total, code\_mass. **Code-Qualität (ESLint+SonarJS):** cc\_loc, cc\_functions, cc\_longest\_function, cc\_avg\_loc\_per\_function, smell\_total, smell\_complexity, smell\_magic\_numbers, smell\_duplication, smell\_code\_quality, coverage\_statements\_pct, coverage\_branches\_pct. **TDD-Disziplin:** cycle\_count, refactorings\_applied, predictions\_correct/total, tests\_passed\_immediately, avg\_red\_seconds, avg\_green\_seconds, avg\_refactor\_seconds.

### 3.3 Bewertungsgrundsätze

- **Korrektheit zuerst:** ein Run mit tests\_passing=false zählt nicht als gleichwertige Lösung.
- **Pro Kata aggregieren:** Workflow×Modell-Tabellen werden ausschließlich pro Kata gebildet.
- **Effekt-Schwelle:** Bei n=1 pro Zelle gelten nur Differenzen mit Faktor  $\geq 2$  oder klar getrennten  $\sigma$ -Bändern als belastbar.

---

## 4. Ergebnisse

### 4.1 RQ-1 — Wirkt der gewählte Workflow auf Code-Qualität, Korrektheit und TDD-Disziplin?

*Datenbasis: 30 Runs · Coverage: 5/5 Zellen (100 %) bei min\_replicates=3.*

**Befunde** (sortiert: haltbar → revidiert → verworfen → nicht-prüfbar):

- [OK] **F-1.10** — Funktionslänge bestätigt v3-Phantom-TDD
- [OK] **F-1.11** — Prediction-Trefferquote bei v4 und v5 vergleichbar hoch
- [OK] **F-1.3** — v3 macht kein echtes TDD
- [OK] **F-1.4** — Speed-Ranking konstant,  $v4 \approx 14 \times v1$
- [OK] **F-1.8** — TDD-Disziplin-Bänder (Refactorings, Predictions) belastbar
- [OK] **F-1.9** — cc\_longest\_function trennt TDD-mit-Refactor scharf vom Rest
- [!] **F-1.1** — TDD  $\Rightarrow$  einfacherer Code, aber nur mit echtem Refactor
- [!] **F-1.2** — v2-iterative und v3-basic-tdd teilen den schlechtesten smell\_total
- [!] **F-1.5** — v4 hat den höchsten "sofort-grün"-Anteil ( $\neq$  schlechte Disziplin)
- [!] **F-1.6** — v5 ist code-kompakter als v4
- [!] **F-1.7** — Workflow-Ranking ohne Thinking

Der Workflow trennt scharf entlang des Refactor-Schritts: v4 und v5 halten cc\_longest\_function bei  $\sim 15$ – $17$  und cc\_avg\_loc\_per\_function bei  $\sim 6$ – $7$ , während v1/v2/v3 bei  $\sim 27$ – $31$  bzw.  $\sim 10$ – $13$  liegen (F-1.9, F-1.10). v3 schreibt im Mittel gleich lange Funktionen wie das non-TDD-v2 — TDD-Etikett ohne Refactor-Schritt kauft also keine Code-Einfachheit (F-1.3, F-1.10). Korrektheit ist auf Opus-no-thinking + game-of-life über alle Workflows 100 %, Speed-Ranking  $v1 \leq v3 \leq v2 < v5 \ll v4$  mit Faktor  $v4/v1 \approx 15.6 \times$  ist stabil (F-1.4). Caveat: das Smell-Ranking selbst ist auf game-of-life-Daten begrenzt — v1/v2/v3-Mittelwerte für smell\_total haben  $\sigma \approx 1$ – $2$ , also keine harte Trennung über Katas hinweg (F-1.2 [!]). Details: research/RQ-1-workflow-effect/findings.md.

## 4.2 RQ-2 — Wirkt Prompt-Stil (prose / example-mapping / user-story) auf Code-Qualität und Korrektheit?

Datenbasis: 13 Runs · Coverage: 3/3 Zellen (100 %) bei min\_replicates=3.

**Befunde** (sortiert: haltbar → revidiert → verworfen → nicht-prüfbar):

- [OK] **F-2.1** — Prompt-Stil hat keinen sichtbaren Pass-Rate-Effekt
- ☒ **F-2.2** — Code-Mass und cc\_longest deuten auf user-story als knappstes Format

Auf game-of-life × v4-exact-subagents × Opus liefern alle drei Prompt-Stile 100 % tests\_passing (F-2.1) — die Hypothese “example-mapping erhöht Pass-Rate” ist für gesättigte Settings nicht prüfbar. Die qualitativen Unterschiede in code\_mass (user-story 149 vs. example-mapping 169) und cc\_longest\_function (8.0 vs. 15.2) deuten auf user-story als knappstes Format, sind aber bei n=3 und  $\sigma \approx 5\text{--}26$  ein potenzielles Stichproben-Artefakt und damit als [X] nicht prüfbar markiert (F-2.2). Replikation mit n≥6 und Workflow-Variation (v3, v5) ist die offene Aktion. Details: research/RQ-2-prompt-style/findings.md.

## 4.3 RQ-3 — Wirken Modell-Klasse (Opus / Sonnet / Haiku) und Thinking-Mode auf Output-Qualität und Effizienz?

Datenbasis: 18 Runs · Coverage: 5/5 Zellen (100 %) bei min\_replicates=3.

**Befunde** (sortiert: haltbar → revidiert → verworfen → nicht-prüfbar):

- [OK] **F-3.1** — Modell-Klasse korreliert mit Code-Qualität (Opus < Sonnet < Haiku)
- [OK] **F-3.2** — Extended Thinking hilft bei cc\_longest, nicht universell bei smell, gar nicht bei Pass-Rate
- [OK] **F-3.3** — Pass-Rate-Sättigung auf v4-exact-subagents
- [OK] **F-3.4** — Haiku ist token-teuerster

Das Code-Mass-Ranking Opus < Sonnet < Haiku ist auf game-of-life × v4 × example-mapping mit n=3–6 robust: Opus liefert ~37 % weniger LoC als Haiku (169 vs. 273) bei deutlich kürzerem cc\_longest\_function (8.7 vs. 19.7) (F-3.1). Thinking reduziert cc\_longest\_function bei Opus deutlich (8.7 vs. 15.2), beeinflusst Pass-Rate aber gar nicht (F-3.2, F-3.3 — alle 18 Runs grün). Haiku ist mit 3.81M Tokens trotz nominell günstigerem Preis der token-teuerste Run, ~50 % über Opus-no-thinking (F-3.4). Caveat: Pass-Rate-Differenzierung zwischen Modellen ist auf v4 + game-of-life gesättigt; H1 (“Opus > Sonnet > Haiku robuster”) muss auf schwierigere Settings (mars-rover, v3-prose) verlagert werden. Details: research/RQ-3-model-and-thinking/findings.md.

## 4.4 RQ-4 — Profitieren schwächere Modelle stärker von strikteren Workflows als starke?

Datenbasis: 48 Runs · Coverage: 12/12 Zellen (100 %) bei min\_replicates=3.

**Befunde** (sortiert: haltbar → revidiert → verworfen → nicht-prüfbar):

- [OK] **F-4.5** — v4-exact-subagents minimiert cc\_longest universell
- [OK] **F-4.6** — Modell-spezifisches Code-Volumen-Profil unter TDD
- [!] **F-4.1** — Pass-Rate ist modell-abhängig: Haiku scheitert nur in v5
- [!] **F-4.2** — Bester Workflow hängt vom Thinking-Modus ab
- [!] **F-4.4** — TDD verkleinert Modell-Abstand teilweise
- ☒ **F-4.3** — Thinking-Bonus auf v4 Mass-Reduktion

v4-exact-subagents erzeugt für **alle** drei Modelle die kürzeste längste Funktion: Haiku 44.7 → 19.7 (−56 %), Sonnet 27.0 → 14.0 (−48 %), Opus 29.8 → 15.2 (−49 %) — der Subagent-Refactor-Schritt ist also der universellste Hebel zur Funktion-Komplexitäts-Reduktion (F-4.5). Code-Volumen reagiert dagegen modellabhängig: Opus ist bezüglich code\_mass über v3/v4/v5 nahezu workflow-invariant (157–169), während Haiku in v4 aufbläht (273) und in v5 entweder kollabiert oder kompakt wird (n=2,  $\sigma=133$ ) (F-4.6). Caveat: v5 (single-context) puffert Fehler bei Haiku nicht ab — 1/3 Run scheitert auf game-of-life-example-mapping; v3/v4 sind dort 100 % grün (F-4.1 [!]). Der ältere Befund “Haiku

scheitert in v4/v5" ist mit  $n \geq 3$  nur noch für v5 haltbar. Details: [research/RQ-4-workflow-model-interaction/findings.md](#).

#### 4.5 RQ-5 — Wie groß ist die Run-zu-Run-Varianz innerhalb identischer Zellen?

Datenbasis: 50 Runs · Coverage: 11/11 Zellen (100 %) bei  $\text{min\_replicates}=3$ .

**Befunde** (sortiert: haltbar → revidiert → verworfen → nicht-prüfbar):

- [OK] **F-5.1** — `tests_passing` ist deterministisch ( $\sigma \approx 0$ )
- [OK] **F-5.2** — `code_mass` zeigt mittleres Rauschen,  $\sigma \sim 7\text{--}26$
- [OK] **F-5.3** — `cc_longest_function`  $\sigma \sim 3\text{--}8$ , mit hohen Outliers
- [OK] **F-5.5** — `duration_seconds` ist hochstabil bei v1/v2/v3, sehr unruhig bei v4
- [!] **F-5.4** — `smell_total`  $\sigma \sim 0.5\text{--}2$ , korreliert mit  $\mu$
- **F-5.6** — Konsequenzen für andere RQs

`tests_passing` ist auf Opus + game-of-life über alle 50 Runs deterministisch (50/50 grün,  $n=1$  reicht für diese Metrik) (F-5.1). `code_mass` ist die rauschigste robuste Metrik mit  $\sigma \sim 7\text{--}26$  und v4/user-story als unruhigste Zelle (F-5.2); `cc_longest_function` zeigt bimodale Verteilungen mit Outliers  $> 25$  trotz Min-Werten von 2 (F-5.3); `duration_seconds` skaliert  $\sigma$  proportional zu  $\mu$  — v4/prose hat Range 533–995 s (F-5.5). Konsequenz für die Methodik:  $n \geq 3$  als Mindest-Schwelle für `code_mass/smell_total`,  $n \geq 6$  für `cc_longest`-Vergleiche und v4-Zellen. Caveat: F-5.1 gilt nur für Opus; auf Haiku ist `tests_passing` wegen RQ-4 F-4.1 nicht stabil — schwächere Modelle brauchen auch für Korrektheit Replikation. Details: [research/RQ-5-run-stability/findings.md](#).

---

### 5. Cross-RQ-Synthese

1. **Refactor-Schritt schlägt Modell-Wahl bei Funktions-Komplexität.** RQ-1 zeigt, dass v4/v5 `cc_longest_function` auf  $\sim 15\text{--}17$  senken, während v1/v2/v3 bei  $\sim 27\text{--}31$  liegen (F-1.9). RQ-4 bestätigt das modellweit: v4 minimiert `cc_longest` für Haiku, Sonnet und Opus gleichermaßen mit  $-48$  bis  $-56$  % (F-4.5). Wer Funktionslänge optimieren will, sollte zuerst den Workflow strikt machen — der Hebel ist bei schwachen Modellen sogar größer als bei starken.
2. **v4 ist der Komplexitäts-Killer, v5 der Sweet Spot.** v4 erreicht die niedrigsten Smell-Werte (F-1.7, F-3.2), aber kostet  $15\times$  mehr Wallclock als v1 (F-1.4) und ist die varianz-anfälligste Zelle (F-5.2, F-5.5). v5 erzielt 90 % der Smell-Reduktion zu  $\sim 45$  % der v4-Dauer und höherer Stabilität — bricht aber bei Haiku auf 67 % Pass-Rate (F-4.1). Single-context-Workflows sind für starke Modelle die effizienteste Wahl, für schwache Modelle ein Risiko.
3. **Modell-Wahl entkoppelt Komplexität und Volumen unter TDD.** RQ-3 stellt das Ranking Opus < Sonnet < Haiku für `code_mass` und `cc_longest` fest (F-3.1). RQ-4 verfeinert: TDD verkleinert den Modell-Abstand bei `cc_longest` deutlich (Haiku 19.7 vs. Opus 15.2 in v4 — fast gleichauf), vergrößert ihn aber bei `smell_total` (Opus 2.5, Haiku 5.3 in v4) (F-4.4, F-4.6). "TDD nivelliert alles" ist also zu pauschal — Komplexität pro Funktion wird modellunabhängig, Code-Sauberkeit bleibt Modell-getrieben.
4. **Pass-Rate-Sättigung verlagert die interessanten Hypothesen.** RQ-2 (F-2.1), RQ-3 (F-3.3) und RQ-4 zeigen alle, dass Pass-Rate auf game-of-life + v4 + Opus auf 100 % gesättigt ist. Hypothesen, die Korrektheits-Differenzierung zwischen Prompt-Stilen oder Modellen voraussetzen, brauchen härtere Settings (mars-rover, v3-prose, Haiku). Solange wir auf game-of-life + Opus + v4 testen, lernen wir nur über Code-Qualität, nicht über Robustheit.
5. **Token-Effizienz und Modell-Größe entkoppeln.** RQ-3 F-3.4 zeigt: Haiku verbraucht  $2.3\times$  mehr Tokens als Opus auf identischer Zelle. RQ-1 F-1.4 zeigt: v4 ist  $15\times$  langsamer als v1. Kosten-Effizienz ergibt sich also nicht aus billigem Modell + striktem Workflow, sondern aus starkem Modell + mittel-striktem Workflow (v5 + Opus) — eine Konstellation, die in keiner einzelnen RQ explizit gemacht wird, sondern nur aus der Kombination sichtbar ist.

## 6. Caveats — revidierte / verworfene / nicht-prüfbare Befunde

### [!] Revidiert

- **F-1.1** (RQ-1) — TDD  $\Rightarrow$  einfacherer Code, aber nur mit echtem Refactor · findings
- **F-1.2** (RQ-1) — v2-iterative und v3-basic-tdd teilen den schlechtesten `smell_total` · findings
- **F-1.5** (RQ-1) — v4 hat den höchsten “sofort-grün”-Anteil ( $\neq$  schlechte Disziplin) · findings
- **F-1.6** (RQ-1) — v5 ist code-kompakter als v4 · findings
- **F-1.7** (RQ-1) — Workflow-Ranking ohne Thinking · findings
- **F-4.1** (RQ-4) — Pass-Rate ist modell-abhängig: Haiku scheitert nur in v5 · findings
- **F-4.2** (RQ-4) — Bester Workflow hängt vom Thinking-Modus ab · findings
- **F-4.4** (RQ-4) — TDD verkleinert Modell-Abstand teilweise · findings
- **F-5.4** (RQ-5) — `smell_total`  $\sigma \sim 0.5-2$ , korreliert mit  $\mu$  · findings

### [X] Nicht prüfbar

- **F-2.2** (RQ-2) — Code-Mass und `cc_longest` deuten auf user-story als knappstes Format · findings
  - **F-4.3** (RQ-4) — Thinking-Bonus auf v4 Mass-Reduktion · findings
- 

## 7. Limitierungen

- Nur Anthropic-Modelle (Opus 4.7, Sonnet 4.6, Haiku 4.5) — keine Aussagen zu GPT-, Gemini- oder Open-Source-Modellen.
  - Nur synthetische Katas (game-of-life, mars-rover) — Übertragbarkeit auf produktive Codebases mit existierender Architektur, Legacy-Code oder Mehr-Datei-Kontext nicht geprüft.
  - Nur TypeScript + Vitest + ESLint+SonarJS — Code-Qualitäts-Metriken sind sprach- und Tool-spezifisch; andere Sprachen oder Linter würden andere Smell-Profile liefern.
  - Headless ohne HITL — kein Mensch greift bei Fehl-Pfaden ein, Modelle können in Sackgassen laufen, die ein Pair-Programmer abkürzen würde.
  - $n \leq 6$  pro Zelle, in den meisten Zellen  $n=3$  — Effekte unterhalb Faktor 2 oder  $\sigma$ -getrennter Bänder sind statistisch nicht belastbar.
  - mars-rover ist nur in prose erhoben — example-mapping/user-story-Cells fehlen, RQ-2-Aussagen gelten nur für game-of-life.
  - v2 und v1 fehlen für example-mapping/user-story by design (Workflow $\rightarrow$ Prompt-Mapping); Prompt-Stil-Effekte können daher nicht über alle Workflows hinweg verglichen werden.
- 

## 8. Reproduzierbarkeit

Alle Daten und Analyse-Skripte liegen im Repo:

- `research/RQ-*/README.md` — RQ-Definitionen (Frontmatter mit factors/controls/outcomes)
- `research/RQ-*/findings.md` — persistente Befund-Listen
- `experiments/runs/*/metrics.json` — Rohdaten pro Run
- `experiments/aggregate-by-query.py` — RQ-spezifische Aggregation
- `experiments/batch-plan-from-rq.py` — Batch-Plan-Generierung aus RQ-Frontmatter
- `experiments/docker/Dockerfile + run-batch.sh + batch.sh` — Container-Pipeline
- `experiments/analyze-run.sh + analyze_transcript.py` — Run-Analyse

Container-Pin: `claude-code@2.1.107` (siehe `experiments/docker/Dockerfile`).

---

## 9. Files

| Pfad                                                 | Inhalt                                                                                                   |
|------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| research/RQ-1-workflow-effect/findings.md            | RQ-1 — Wirkt der gewählte Workflow auf Code-Qualität, Korrektheit und TDD-Disziplin?                     |
| research/RQ-1-workflow-effect/runs.csv               | RQ-1 aggregierte Run-Metriken                                                                            |
| research/RQ-2-prompt-style/findings.md               | RQ-2 — Wirkt Prompt-Stil (prose / example-mapping / user-story) auf Code-Qualität und Korrektheit?       |
| research/RQ-2-prompt-style/runs.csv                  | RQ-2 aggregierte Run-Metriken                                                                            |
| research/RQ-3-model-and-thinking/findings.md         | RQ-3 — Wirken Modell-Klasse (Opus / Sonnet / Haiku) und Thinking-Mode auf Output-Qualität und Effizienz? |
| research/RQ-3-model-and-thinking/runs.csv            | RQ-3 aggregierte Run-Metriken                                                                            |
| research/RQ-4-workflow-model-interaction/findings.md | RQ-4 — Profitieren schwächere Modelle stärker von strikteren Workflows als starke?                       |
| research/RQ-4-workflow-model-interaction/runs.csv    | RQ-4 aggregierte Run-Metriken                                                                            |
| research/RQ-5-run-stability/findings.md              | RQ-5 — Wie groß ist die Run-zu-Run-Varianz innerhalb identischer Zellen?                                 |
| research/RQ-5-run-stability/runs.csv                 | RQ-5 aggregierte Run-Metriken                                                                            |
| experiments/runs/                                    | Alle Run-Verzeichnisse mit Source, Transcript, Metriken                                                  |